

HR HELPER - ДЕТАЛЬНЫЙ ПЛАН ДЕЙСТВИЙ

Комплексное руководство по исправлению и улучшению системы

Дата создания: 28.10.2025

Версия: 1.0

Статус: Ready for Implementation

Оглавление

1. Анализ текущего состояния
2. Фаза 1: Критические исправления
3. Фаза 2: Планируемые улучшения
4. Фаза 3: Долгосрочные улучшения
5. План тестирования
6. Метрики успеха

Анализ текущего состояния {#анализ}

Работающие компоненты

Система HR Helper включает следующие **работающие** компоненты:

- **AJAX обработка чата** - реализована через `chatajaxhandler` в `views.py`
- **Определение типа команды** - `determine_action_type_from_text()` корректно распознает команды
- **Создание записей** - HRScreening и Invite через Django формы функционируют
- **Интеграция с Huntflow** - через модели и API сервисы работает стабильно
- **Google Calendar/Drive** - создание событий и документов выполняется успешно
- **ChatMessage** - сохранение в БД с различными типами сообщений

Критические проблемы

Проблема №1: Слоты не генерируются

Локация: `vacancy-slots.js`

Симптомы:

- Слоты не отображаются на странице

- Кнопки копирования слотов неактивны
- Console errors: "calendarEvents is not defined"
- Пустые секции текущей и следующей недели

Причины:

- Неправильный порядок загрузки JavaScript файлов
- Глобальные переменные (`calendarEvents`, `vacancyData`, `slotsSettings`) недоступны при инициализации
- Функция `initializeSlots()` вызывается до полной загрузки данных
- Отсутствие валидации доступности данных перед генерацией

Критичность: P0 (блокирует базовую функциональность)

Проблема №2: AJAX чат перезагружает страницу

Локация: `chat_actions.js`, `views.py::chatajaxhandler`

Симптомы:

- После отправки сообщения страница полностью перезагружается
- История чата теряется при отправке
- Пользовательский опыт нарушен

Причины:

- Функция `reloadChat()` явно вызывает `window.location.reload()`
- Backend не возвращает HTML нового сообщения в JSON response
- Отсутствует функция динамического добавления сообщений в DOM

Критичность: P0 (критический UX issue)

Статус: Решение задокументировано в предыдущей инструкции

Проблема №3: Неверный подсчет встреч

Локация: `vacancy-slots.js::calculateAvailableSlots()`

Симптомы:

- Badge отображает неправильное количество встреч
- "Обеды" учитываются как встречи
- События "весь день" включены в подсчет

Причины:

- Неточная фильтрация событий типа lunch/обед

- Badge показывает количество слотов вместо встреч
- Логика подсчета не синхронизирована с отображением

Критичность: P1 (влияет на точность данных)

Проблема №4: Медленный парсер времени

Локация: enhanced_datetime_parser.py

Симптомы:

- Обработка команды /in занимает >3 секунды
- Пользователь ждет ответа системы
- Негативный impact на UX

Причины:

- Отсутствие кеширования результатов парсинга
- Повторная компиляция регулярных выражений при каждом вызове
- Неоптимальные алгоритмы поиска

Критичность: P1 (влияет на производительность)

Фаза 1: Критические исправления {#фаза-1}

Срок: 1-2 недели

Цель: Устранить все P0 и P1 проблемы

Шаг 1: Исправление генерации слотов

Задача 1.1: Правильная загрузка скриптов

Файл: chat_workflow.html

Решение: Реорганизация порядка загрузки

```
<script id="slots-settings-data" type="application/json">
  {{ slots_settings_json|safe }}
</script>

<script id="vacancy-data" type="application/json">
  {{ vacancy_data_json|safe }}
</script>

<script id="calendar-events-data" type="application/json">
  {{ calendar_events_json|safe }}
</script>
```

```

<script>
    window.slotsSettings = null;
    window.vacancyData = null;
    window.calendarEvents = null;
    window.sessionId = "{{ chat_session.id }}";
    window.userWorkHours = {{ user_work_hours_json|safe }};
    window.userMeetingInterval = {{ user_meeting_interval|default:15 }};
</script>

<script src="{% static 'js/vacancy-slots.js' %}" defer></script>
<script src="{% static 'js/copy-buttons.js' %}" defer></script>
<script src="{% static 'js/chat_actions.js' %}" defer></script>

<script defer>
    document.addEventListener('DOMContentLoaded', function() {
        // Парсинг JSON данных
        const slotsEl = document.getElementById('slots-settings-data');
        if (slotsEl) window.slotsSettings = JSON.parse(slotsEl.textContent);

        const vacancyEl = document.getElementById('vacancy-data');
        if (vacancyEl) window.vacancyData = JSON.parse(vacancyEl.textContent);

        const eventsEl = document.getElementById('calendar-events-data');
        if (eventsEl) {
            window.calendarEvents = JSON.parse(eventsEl.textContent);
        } else {
            window.calendarEvents = [];
        }

        // Запуск инициализации
        if (typeof initializeSlots === 'function') {
            initializeSlots();
        }
    });
</script>

```

Эффект: Данные гарантированно доступны перед инициализацией слотов

Задача 1.2: Улучшенная initializeSlots()

Файл: vacancy-slots.js

Ключевые изменения:

- Полная валидация данных перед генерацией
- Детальное логирование для отладки
- Graceful degradation при отсутствии данных
- Проверка наличия DOM элементов

Пример кода:

```

function initializeSlots() {
  console.log('📅 [INIT] Начало инициализации слотов');

  // Валидация данных
  if (!window.calendarEvents) {
    console.warn('⚠️ calendarEvents не определены');
    window.calendarEvents = [];
  }

  if (!Array.isArray(window.calendarEvents)) {
    console.error('❌ calendarEvents не массив!');
    window.calendarEvents = [];
  }

  // Проверка DOM
  const sections = {
    current: document.querySelector('.week-section.current-week'),
    next: document.querySelector('.week-section.next-week')
  };

  if (!sections.current || !sections.next) {
    console.error('❌ Секции слотов не найдены!');
    return;
  }

  // Генерация и отображение
  const currentSlots = generateWeekSlots(0);
  const nextSlots = generateWeekSlots(1);

  updateSlotsDisplay(currentSlots, nextSlots);
  updateLastUpdateTime();

  console.log('✅ [INIT] Инициализация завершена');
}

```

Эффект: Стабильная генерация слотов с информативными логами

Задача 1.3: Backend сериализация

Файл: views.py

Функция: chatworkflow(request)

Ключевые изменения:

- Правильная сериализация календарных событий
- JSON-формат всех данных
- Обработка исключений при загрузке календаря

Пример кода:

```

import json
from datetime import datetime, timedelta

def chatworkflow(request):
    # ... получение пользователя, вакансии, чата ...

    # Загрузка календарных событий
    calendar_events = []
    try:
        oauth_service = GoogleOAuthService(user)
        oauth_account = oauth_service.get_oauth_account()

        if oauth_account:
            calendar_service = GoogleCalendarService(oauth_service)
            start_date = datetime.now()
            end_date = start_date + timedelta(days=14)

            events = calendar_service.get_events(
                oauth_account,
                time_min=start_date,
                time_max=end_date
            )

            for event in events:
                calendar_events.append({
                    'id': event.get('id'),
                    'title': event.get('summary', 'Без названия'),
                    'start': event.get('start', {}).get('dateTime') or
                        event.get('start', {}).get('date'),
                    'end': event.get('end', {}).get('dateTime') or
                        event.get('end', {}).get('date'),
                    'isallday': 'date' in event.get('start', {}),
                })
    except Exception as e:
        print(f"✖ Ошибка загрузки календаря: {e}")

    # Контекст с JSON данными
    context = {
        'slots_settings_json': json.dumps(slots_settings.to_dict()),
        'vacancy_data_json': json.dumps(vacancy_data),
        'calendar_events_json': json.dumps(calendar_events),
        'user_work_hours_json': json.dumps(user_work_hours),
        'user_meeting_interval': user_meeting_interval,
    }

    return render(request, 'googleoauth/chat_workflow.html', context)

```

Эффект: Корректная передача данных от backend к frontend

Шаг 2: Свитчер Интервью/Скрининг

Задача 2.1: HTML UI свитчера

Добавить в Блок 1 страницы:

```
<div>
  <div>
    <div>
      &lt;label class="form-label fw-bold mb-2"&gt;
        &lt;i class="fas fa-tasks me-2"&gt;&lt;/i&gt;Тип встречи
      &lt;/label&gt;

      <div>
        &lt;input type="radio" class="btn-check" name="meetingType"
          id="typeScreening" value="screening" checked&gt;
        &lt;label class="btn btn-outline-success" for="typeScreening"&gt;
          &lt;i class="fas fa-clipboard-list me-2"&gt;&lt;/i&gt;Скрининг
        &lt;/label&gt;

        &lt;input type="radio" class="btn-check" name="meetingType"
          id="typeInterview" value="interview"&gt;
        &lt;label class="btn btn-outline-primary" for="typeInterview"&gt;
          &lt;i class="fas fa-users me-2"&gt;&lt;/i&gt;Интервью
        &lt;/label&gt;
      </div>

      &lt;small class="text-muted d-block mt-2"&gt;
        &lt;i class="fas fa-info-circle me-1"&gt;&lt;/i&gt;
        <span>
          Скрининг: 40-60 мин, интервью: 60-90 мин
        </span>
      &lt;/small&gt;
    </div>
  </div>
</div>
```

Эффект: Визуальный интерфейс для выбора типа встречи

Задача 2.2: JavaScript логика

Новый файл: meeting-type-switcher.js

Функциональность:

- Отслеживание текущего типа встречи
- Обновление длительности при смене типа
- Пересчет слотов
- Custom events для интеграции с другими модулями

Основная логика:

```

const MEETING_CONFIG = {
  screening: {
    defaultDuration: 60,
    info: 'Скрининг: 40-60 мин'
  },
  interview: {
    defaultDuration: 90,
    info: 'Интервью: 60-90 мин'
  }
};

function handleMeetingTypeChange(newType) {
  const config = MEETING_CONFIG[newType];

  // Обновить vacancyData
  if (window.vacancyData) {
    window.vacancyData.duration = config.defaultDuration;
  }

  // Пересчитать слоты
  if (typeof initializeSlots === 'function') {
    initializeSlots();
  }

  // Dispatch event
  document.dispatchEvent(new CustomEvent('meetingTypeChanged', {
    detail: { type: newType, duration: config.defaultDuration }
  }));
}

```

Эффект: Динамическая смена типа встречи без перезагрузки

Шаг 3: Исправление подсчета встреч

Задача 3.1: Улучшенный generateWeekSlots()

Ключевые изменения:

- Правильная фильтрация событий (исключение обедов, событий "весь день")
- Точный подсчет рабочих встреч
- Детальное логирование процесса

Логика фильтрации:

```

const meetingsWithoutLunch = dayEvents.filter(event => {
  const title = event.title.toLowerCase();

  // Исключаем обеды
  const isLunch = title.includes('обед') ||
    title.includes('lunch');
  if (isLunch) return false;
});

```



```

    // Исключаем "весь день"
    if (event.isallday) return false;

    // Исключаем нерабочие события
    const isNonWorking = title.includes('отпуск') ||
        title.includes('vacation');
    if (isNonWorking) return false;

    return true;
});

meetingsCount = meetingsWithoutLunch.length;

```

Эффект: Точный подсчет рабочих встреч в badge

Задача 3.2: Улучшенный createSlotCard()

Добавление badge для встреч:

```

const meetingsBadge = slot.meetingsCount > 0 ?
    `<span>
        &lt;i class="fas fa-calendar-check me-1"&gt;&lt;/i&gt;
        ${slot.meetingsCount}
    </span>` : '';

```

Эффект: Визуальное отображение количества встреч в день

Шаг 4: Оптимизация парсера времени

Задача 4.1: Кеширование и предкомпиляция

Ключевые улучшения:

1. **LRU кеширование** - кеш последних 1024 результатов
2. **Предкомпиляция регулярков** - компиляция один раз при инициализации
3. **Быстрая предпроверка** - раннее отсеивание нерелевантного текста

Пример оптимизации:

```

from functools import lru_cache

class EnhancedDateTimeParser:
    def __init__(self):
        self._compiled_patterns = {}
        self._precompile_patterns()

    def _precompile_patterns(self):
        self._compiled_patterns['time'] = [

```

```

        re.compile(r'(\d{1,2}):(\d{2})', re.IGNORECASE),
        # ... другие паттерны
    ]

    @lru_cache(maxsize=1024)
    def parse(self, text: str) -> Optional[datetime]:
        if not self._has_time_markers(text):
            return None
        # ... основная логика

    def _has_time_markers(self, text: str) -> bool:
        markers = [':', 'ч', 'час', '2025', 'понедельник']
        text_lower = text.lower()
        return any(marker in text_lower for marker in markers)

```

Эффект: Ускорение парсинга в 5-10 раз

Фаза 2: Планируемые улучшения {#фаза-2}

Срок: 2-4 недели после Фазы 1

Команда /del - Система отката

Компоненты:

- State snapshot service для каждого пользователя/чата
- Rollback механизм для Huntflow
- UI отображения отмененных изменений

Формат сообщения:

✖ Отменено: HR-скрининг
 Кандидат: Иванов Иван
 Вакансия: Frontend Engineer

Изменения:

- Грейд: Middle → (удалено)
- Зарплата: 2000 USD → (удалено)
- Статус в Huntflow: откачен

Команда /t - Техническое интервью

Отличия от /in:

- Без создания scorecard
- Другие временные правила (60-90 мин)
- Подтягивание предыдущих участников
- Отображение текущего статуса из Huntflow

Улучшение системы уведомлений

Замена alert() на:

- Toast-уведомления (Bootstrap или custom)
- Прогресс-индикаторы для длительных операций
- Анимации появления/исчезновения

Пример toast:

```
function showToast(message, type = 'success') {
  const toast = document.createElement('div');
  toast.className = `toast-notification alert alert-${type}`;
  toast.textContent = message;
  document.body.appendChild(toast);

  setTimeout(() => toast.classList.add('show'), 100);
  setTimeout(() => {
    toast.classList.remove('show');
    setTimeout(() => toast.remove(), 300);
  }, 5000);
}
```

Фаза 3: Долгосрочные улучшения {#фаза-3}

Срок: 1-3 месяца

AI-назначение интервьюеров

Факторы анализа:

- Специализация кандидата vs экспертиза интервьюера
- Загруженность интервьюеров (календарь)
- Предыдущий опыт интервьюирования
- Временные предпочтения команды

Алгоритм:

1. Анализ профиля кандидата (технологии, опыт)
2. Поиск интервьюеров с подходящей экспертизой
3. Проверка загруженности через Google Calendar API
4. Ранжирование по score: экспертиза × доступность
5. Автоматическое предложение топ-3

Интеграция с календарями интервьюеров

Функциональность:

- Проверка свободных слотов в календарях команды
- Автоматическое предложение оптимального времени
- Синхронизация с Outlook, Google Calendar

API интеграция:

- Google Calendar API (уже есть)
- Microsoft Graph API (для Outlook)
- CalDAV протокол (универсальный)

Аналитика и метрики

Dashboards:

1. Производительность:

- Среднее время обработки команд
- Количество созданных скринингов/интервью
- Success rate операций

2. HR метрики:

- Конверсия по грейдам
- Time-to-hire
- Эффективность рекрутеров

3. Технические метрики:

- API response times
- Error rates
- Cache hit rates

План тестирования {#тестирование}

Тестирование Фазы 1

Неделя 1: Слоты

Чеклист:

- [] Слоты генерируются для текущей недели
- [] Слоты генерируются для следующей недели
- [] calendarEvents загружаются корректно

- ☐ Кнопки копирования активны при наличии слотов
- ☐ Кнопки неактивны при отсутствии слотов
- ☐ Нет ошибок в консоли браузера
- ☐ Обновление слотов работает

Тест-кейсы:

1. Пустой календарь → все слоты свободны
2. Полностью занятый календарь → нет слотов
3. Частично занятый → корректные интервалы
4. С обедами → обеды исключены из подсчета

Неделя 2: Свитчер

Чеклист:

- ☐ Свитчер отображается корректно
- ☐ Переключение меняет тип встречи
- ☐ Слоты пересчитываются после смены
- ☐ Длительность обновляется корректно
- ☐ Info текст меняется
- ☐ Event meetingTypeChanged срабатывает

Тест-кейсы:

1. Скрининг → Интервью: длительность 60 → 90
2. Интервью → Скрининг: длительность 90 → 60
3. Смена типа пересчитывает слоты
4. Сохранение выбора при навигации (опционально)

Неделя 3: Подсчет встреч

Чеклист:

- ☐ meetingsCount точный для каждого дня
- ☐ Badge отображает правильное число
- ☐ Обеда исключены из подсчета
- ☐ События "весь день" исключены
- ☐ Нерабочие события (отпуск) исключены

Тест-кейсы:

1. День с 3 встречами + обед → badge показывает 3

2. День с событием "весь день" → не учитывается
3. День с отпуском → не учитывается
4. Пустой день → badge 0 или не отображается

Неделя 4: Парсер времени

Чеклист:

- [] Парсинг < 1 секунды
- [] Кеш работает (повторный парсинг мгновенный)
- [] Все форматы распознаются
- [] Некорректный ввод обрабатывается gracefully

Тест-кейсы:

1. "15.09.2025 14:30" → корректный datetime
2. "завтра в 14:00" → корректный datetime
3. "понедельник 15ч" → корректный datetime
4. "абракадабра" → None или ошибка

Unit-тесты

Python (backend)

```
class TestEnhancedDateTimeParser(TestCase):
    def setUp(self):
        self.parser = get_parser()

    def test_absolute_date_time(self):
        result = self.parser.parse("15.09.2025 14:30")
        self.assertEqual(result.day, 15)
        self.assertEqual(result.month, 9)
        self.assertEqual(result.year, 2025)
        self.assertEqual(result.hour, 14)
        self.assertEqual(result.minute, 30)

    def test_relative_date(self):
        result = self.parser.parse("завтра в 14:00")
        expected = datetime.now() + timedelta(days=1)
        self.assertEqual(result.day, expected.day)
        self.assertEqual(result.hour, 14)

    def test_invalid_input(self):
        result = self.parser.parse("invalid text")
        self.assertIsNone(result)

    def test_caching(self):
```

```

# Первый вызов
start = time.time()
self.parser.parse("15.09.2025 14:30")
first_call = time.time() - start

# Второй вызов (из кеша)
start = time.time()
self.parser.parse("15.09.2025 14:30")
second_call = time.time() - start

# Должно быть намного быстрее
self.assertLess(second_call, first_call * 0.1)

```

JavaScript (frontend)

```

describe('Meeting Type Switcher', () => {
  beforeEach(() => {
    // Setup DOM
    document.body.innerHTML = `
      <input type="radio" id="typeScreening" checked="" />
      <input type="radio" id="typeInterview" />
    <span></span>
    `;
  });

  test('should initialize with screening type', () => {
    expect(getCurrentMeetingType()).toBe('screening');
  });

  test('should change to interview type', () => {
    const interviewRadio = document.getElementById('typeInterview');
    interviewRadio.checked = true;
    interviewRadio.dispatchEvent(new Event('change'));

    expect(getCurrentMeetingType()).toBe('interview');
  });

  test('should update duration on type change', () => {
    window.vacancyData = { duration: 60 };

    const interviewRadio = document.getElementById('typeInterview');
    interviewRadio.checked = true;
    interviewRadio.dispatchEvent(new Event('change'));

    expect(window.vacancyData.duration).toBe(90);
  });
});

```

Метрики успеха {#метрики}

Технические метрики

Производительность

Метрика	Текущее	Целевое	Критерий
Время загрузки слотов	N/A	< 2 сек	P0
Время парсинга даты	~3-5 сек	< 1 сек	P1
Время отправки AJAX	~1 сек	< 500 мс	P2
FCP (First Contentful Paint)	?	< 1.5 сек	P2

Надежность

Метрика	Текущее	Целевое	Критерий
Успешность генерации слотов	0%	99%	P0
Успешность AJAX запросов	~95%	99%	P0
Точность подсчета встреч	~70%	100%	P1
Cache hit rate (парсер)	0%	>80%	P1

Бизнес-метрики

Использование

Метрика	Базовая	Целевая	Период
Скринингов в день	?	+50%	1 месяц
Инвайтов в день	?	+30%	1 месяц
Активных пользователей	?	100% команды	2 месяца
NPS (Net Promoter Score)	?	>8/10	3 месяца

Эффективность

Метрика	До	После	Улучшение
Время на скрининг	~10 мин	~5 мин	-50%
Время на инвайт	~15 мин	~7 мин	-53%
Ошибок в данных	~5%	<1%	-80%

Заключение

Данный план действий обеспечивает **систематический подход** к исправлению критических проблем и внедрению улучшений в систему HR Helper.

Приоритеты

Немедленно (Фаза 1):

1. Исправить генерацию слотов
2. Добавить свитчер типов встреч
3. Исправить подсчет встреч
4. Оптимизировать парсер времени

Ближайшие 2-4 недели (Фаза 2):

1. Команда /del
2. Команда /t
3. Улучшенные уведомления

Долгосрочно (Фаза 3):

1. AI-функции
2. Расширенная интеграция
3. Аналитика

Критерии успеха

Фаза 1 считается **успешно завершенной**, когда:

- ✔ Слоты генерируются стабильно для обеих недель
- ✔ Свитчер меняет тип встречи без ошибок
- ✔ Подсчет встреч точный (100%)
- ✔ Парсер времени работает < 1 сек
- ✔ AJAX чат работает без перезагрузки
- ✔ Все unit-тесты проходят

Следующий шаг: Приступить к реализации Шага 1.1